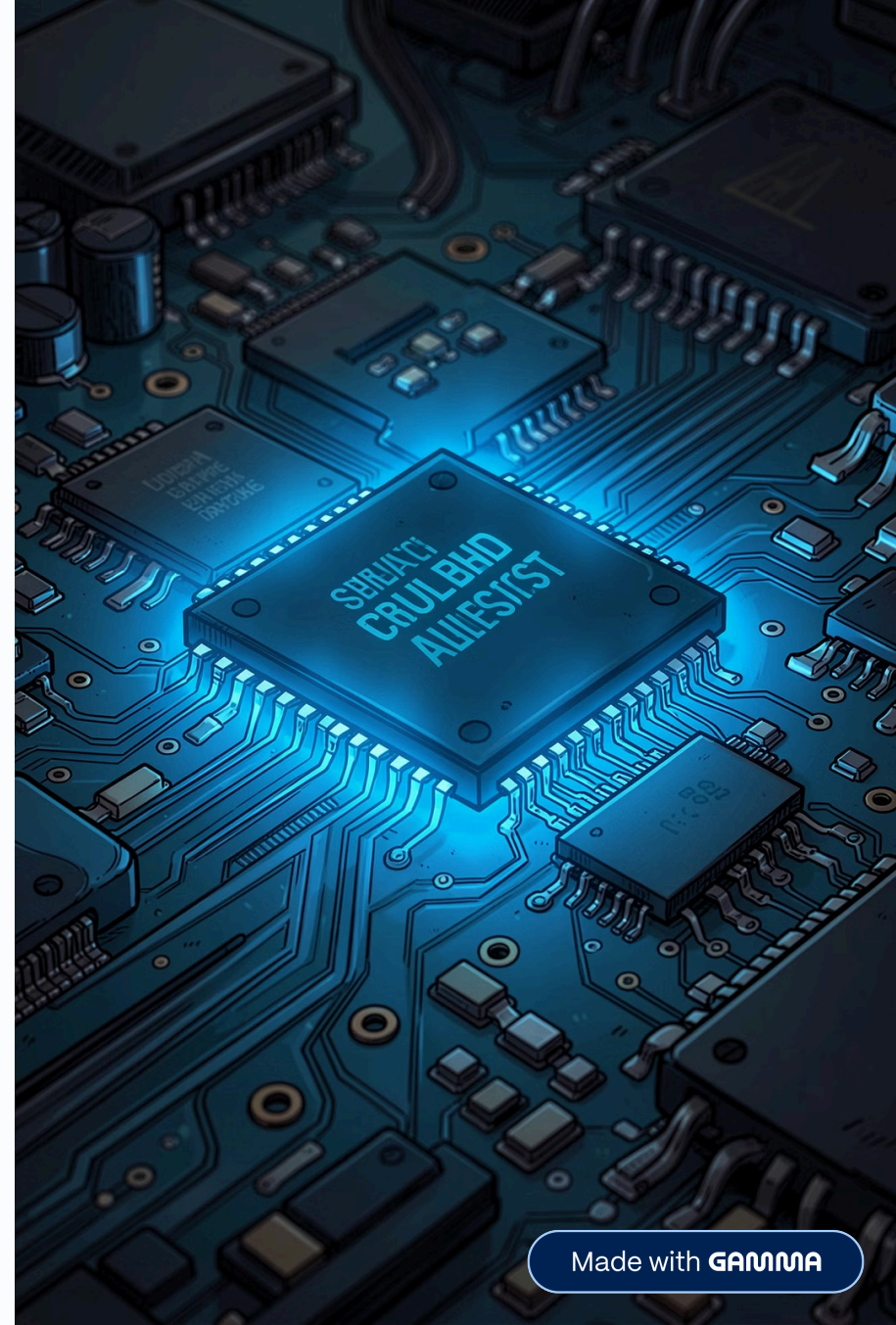


Gerenciamento de Memória

Aula 13 — A Arte de Otimizar o Espaço de Trabalho do Computador

SISTEMAS OPERACIONAIS

MEMÓRIA RAM



O Desafio: Memória RAM é um Recurso Valioso e Escasso

A memória RAM é o "**espaço de trabalho**" do sistema operacional — é onde processos ativos, dados em uso e o próprio SO residem durante a execução. Sem gerenciamento adequado, processos competem por esse espaço limitado e o sistema entra em colapso.

→ Demanda Crescente

Dados multimídia, navegadores com múltiplas abas e aplicações paralelas exigem cada vez mais memória.

→ Custo e Energia

Aumentar a RAM física nem sempre é viável — hardware tem limite de slots, custo elevado e consumo energético adicional.

→ Papel do SO

O sistema operacional deve gerenciar a alocação de forma inteligente, garantindo desempenho e estabilidade.

Por que importa?

Um gerenciamento ineficiente de memória causa lentidão, travamentos e até falhas críticas do sistema. É uma das responsabilidades centrais de qualquer SO moderno.

Swapping: Trocando Processos para Ganhar Espaço

O **swapping** é uma técnica que permite ao sistema operacional mover processos temporariamente ociosos da RAM para o disco rígido (**swap-out**) e restaurá-los quando voltam a ser necessários (**swap-in**). Isso cria a ilusão de uma memória maior do que a fisicamente disponível.

  **Analogia:** Como um chef que guarda ingredientes menos utilizados em um armário para liberar espaço valioso na bancada de trabalho — tudo está disponível, mas só o necessário fica à mão.

Swap-Out

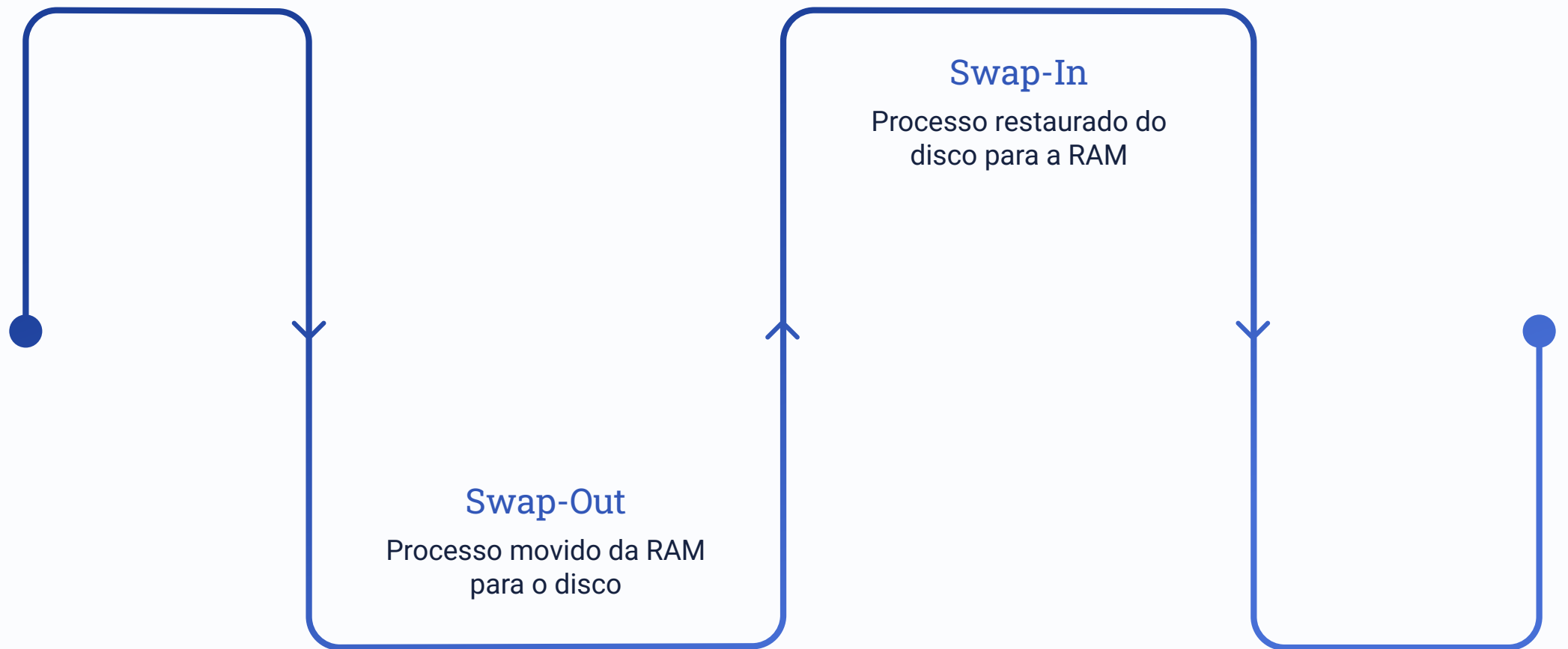
Processo ocioso é movido da RAM para uma área reservada no disco (partição de swap).

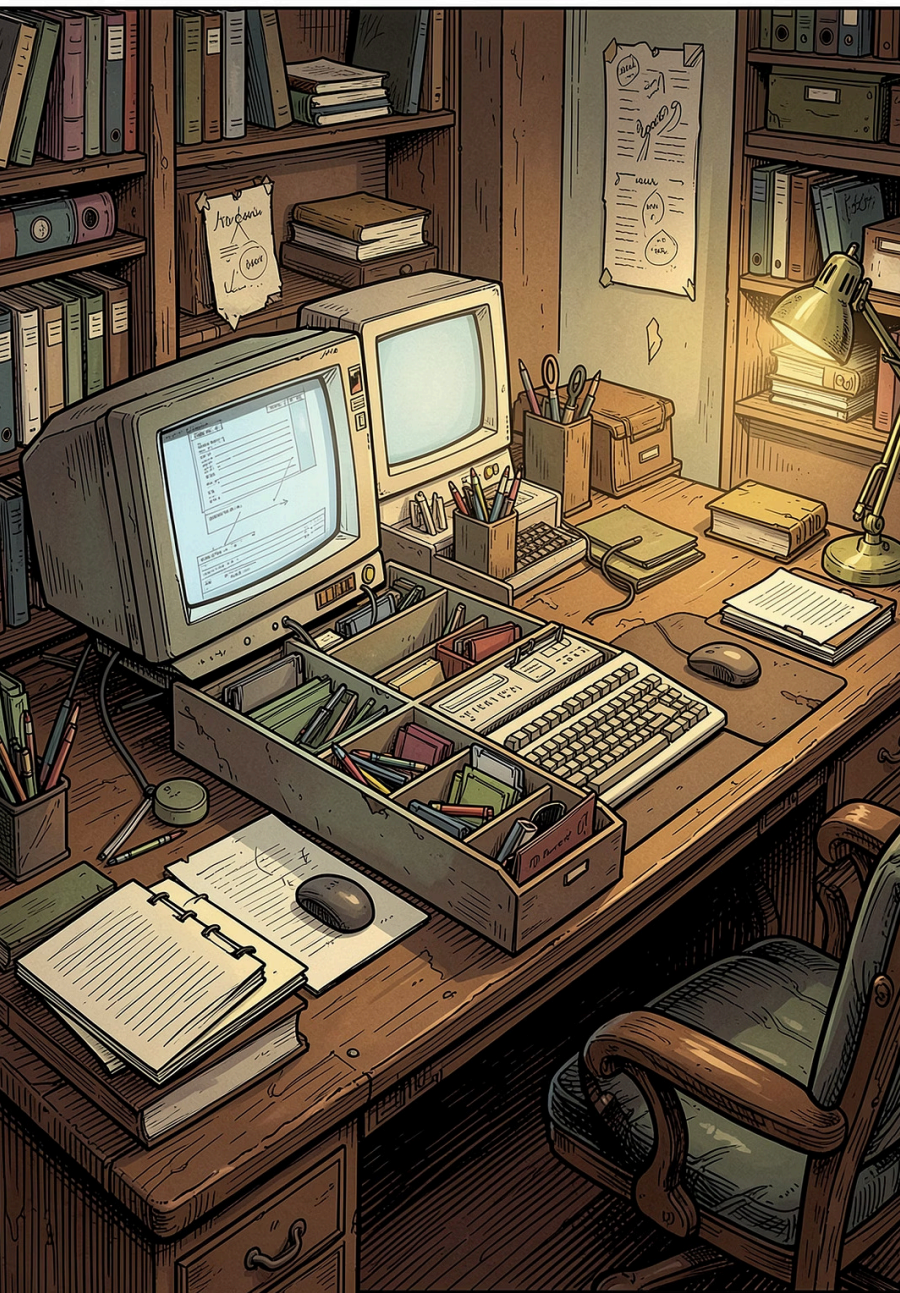
Swap-In

Quando o processo volta a ser necessário, é carregado novamente na RAM para continuar a execução.

Contexto Histórico

Técnica amplamente utilizada até os anos 90. Hoje menos comum em sistemas de uso geral, substituída por técnicas mais sofisticadas.





Alocação Contígua: O Método Simples (e Limitado)

Na **alocação contígua**, cada processo ocupa um bloco contínuo e ininterrupto de memória. O SO divide a RAM em partições — fixas ou variáveis — e atribui uma a cada processo. É o modelo mais intuitivo, mas carregado de limitações práticas.

Partições Fixas

A memória é dividida em blocos de tamanho predefinido. Simples de implementar, mas desperdiça espaço quando o processo é menor que a partição.

Partições Variáveis

O tamanho da partição é ajustado ao tamanho do processo. Reduz o desperdício interno, mas cria buracos externos ao longo do tempo.

Exemplo Real

Utilizado no MS-DOS e em sistemas embarcados com aplicação única. Não suporta multiprogramação robusta.

Fragmentação: O Inimigo do Espaço Livre

Fragmentação Externa

Ocorre quando há memória livre suficiente no total, mas ela está **dispersa em blocos pequenos e não contíguos**, impedindo a alocação de novos processos maiores. O espaço existe, mas é inutilizável no estado atual.

Fragmentação Interna

Ocorre quando um processo recebe uma partição maior do que realmente precisa. O **espaço excedente dentro da partição é desperdiçado** pois não pode ser usado por outros processos.

Solução: Compactação

O SO pode reorganizar os processos na memória para juntar os fragmentos livres em um bloco contíguo maior. Porém, é uma operação custosa em termos de tempo de processamento.

Analogia Prática

Imagine ter vários pedaços pequenos de papel espalhados pela mesa. Individualmente, nenhum é grande o suficiente para imprimir um documento inteiro — mas somados, o espaço total seria mais do que suficiente.

Esse é exatamente o problema da fragmentação externa: **memória disponível, porém inutilizável.**

Paginação: Dividindo a Memória em Pedacos Iguais



Conceito Básico

A RAM é dividida em **frames** de tamanho fixo e os processos em **páginas** do mesmo tamanho. Páginas são carregadas em frames disponíveis, não precisando ser contíguas na memória física.



Page-In / Page-Out

O SO gerencia a transferência de páginas entre RAM e disco. Quando uma página acessada não está na RAM (**page fault**), ela é carregada automaticamente pelo SO.



Hardware: MMU

A **Memory Management Unit** traduz endereços lógicos (do processo) em endereços físicos (na RAM) usando a **tabela de páginas**, tornando o processo transparente para o programador.



Proteção

Cada entrada na tabela de páginas contém **bits de controle** (leitura, escrita, execução) que impedem um processo de acessar indevidamente páginas pertencentes a outro processo.

Segmentação: A Visão do Programador

Enquanto a paginação é invisível ao programador, a **segmentação reflete diretamente a estrutura lógica do programa**. A memória é dividida em segmentos com significado semântico real — cada um com tamanho variável e atributos próprios de proteção.

☐ Segmentos típicos: **Código** (text), **Dados** (data), **Pilha** (stack) e **Heap**.

1 Estrutura Lógica

Cada segmento representa uma parte funcional do programa. O endereço é composto por (**número do segmento, deslocamento**), tornando o modelo intuitivo para o desenvolvedor.

2 Compartilhamento e Proteção

Segmentos podem ser **compartilhados entre processos** (ex: bibliotecas de código) e protegidos individualmente com permissões específicas de leitura e escrita.

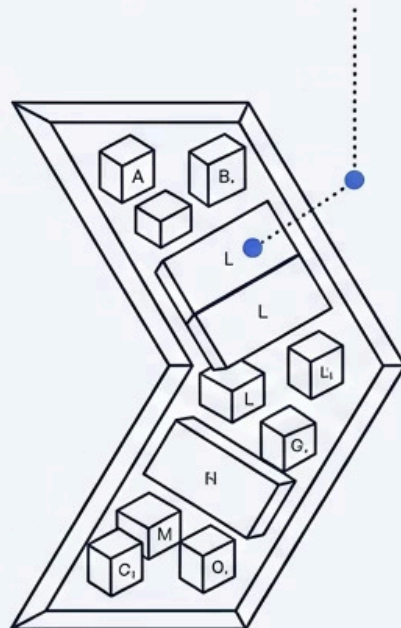
3 Desafio: Fragmentação

Como os segmentos têm tamanhos variáveis, a alocação e liberação sucessiva gera **fragmentação externa**, exigindo estratégias de gerenciamento cuidadosas.

Segmentação Paginada: O Melhor dos Dois Mundos?

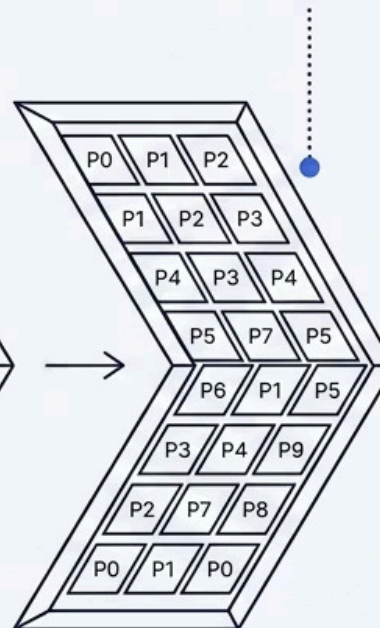
A **segmentação paginada** combina as vantagens de ambas as abordagens, mitigando suas fraquezas individuais.

Segmentação



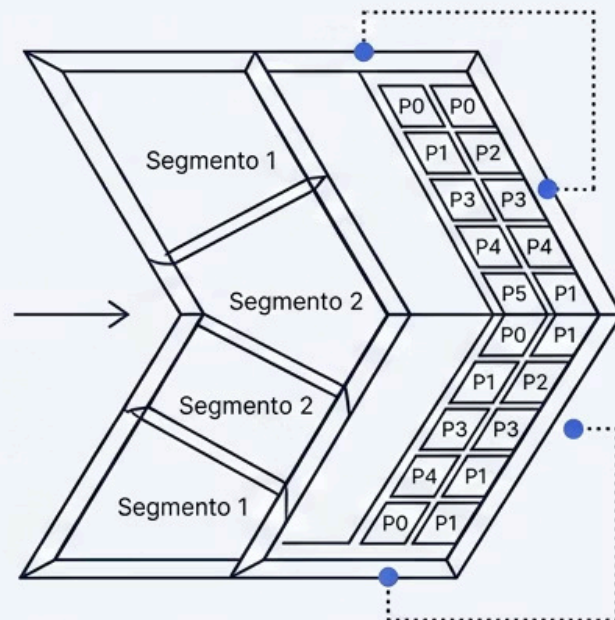
Divisão lógica do programa

Paginação



Divisão em páginas fixas

Segmentação Paginada



Segmentação Paginada
Visão lógica, tamanho granular, sem fragmentação externa

✓ **Vantagem:** A visão lógica do programador (segmentos) é mantida, enquanto a gestão física usa páginas — eliminando a fragmentação externa.

⚠ **Custo:** A complexidade do hardware e das estruturas de dados aumenta, pois é necessário gerenciar tanto a tabela de segmentos quanto as tabelas de páginas de cada segmento.

Proteção de Memória: Guardando os Segredos de Cada Processo

A **proteção de memória** é um requisito fundamental para a estabilidade, segurança e isolamento dos processos em um sistema multitarefa. Sem ela, um processo com falha poderia corromper a memória de outro — ou até do próprio SO.



Bits de Proteção

Cada página ou segmento possui bits que definem permissões: leitura, escrita e execução. Tentativas de violação geram uma exceção de hardware (trap).



Isolamento de Processos

Cada processo enxerga apenas seu próprio espaço de endereçamento virtual. O acesso à memória de outro processo é fisicamente bloqueado pela MMU.

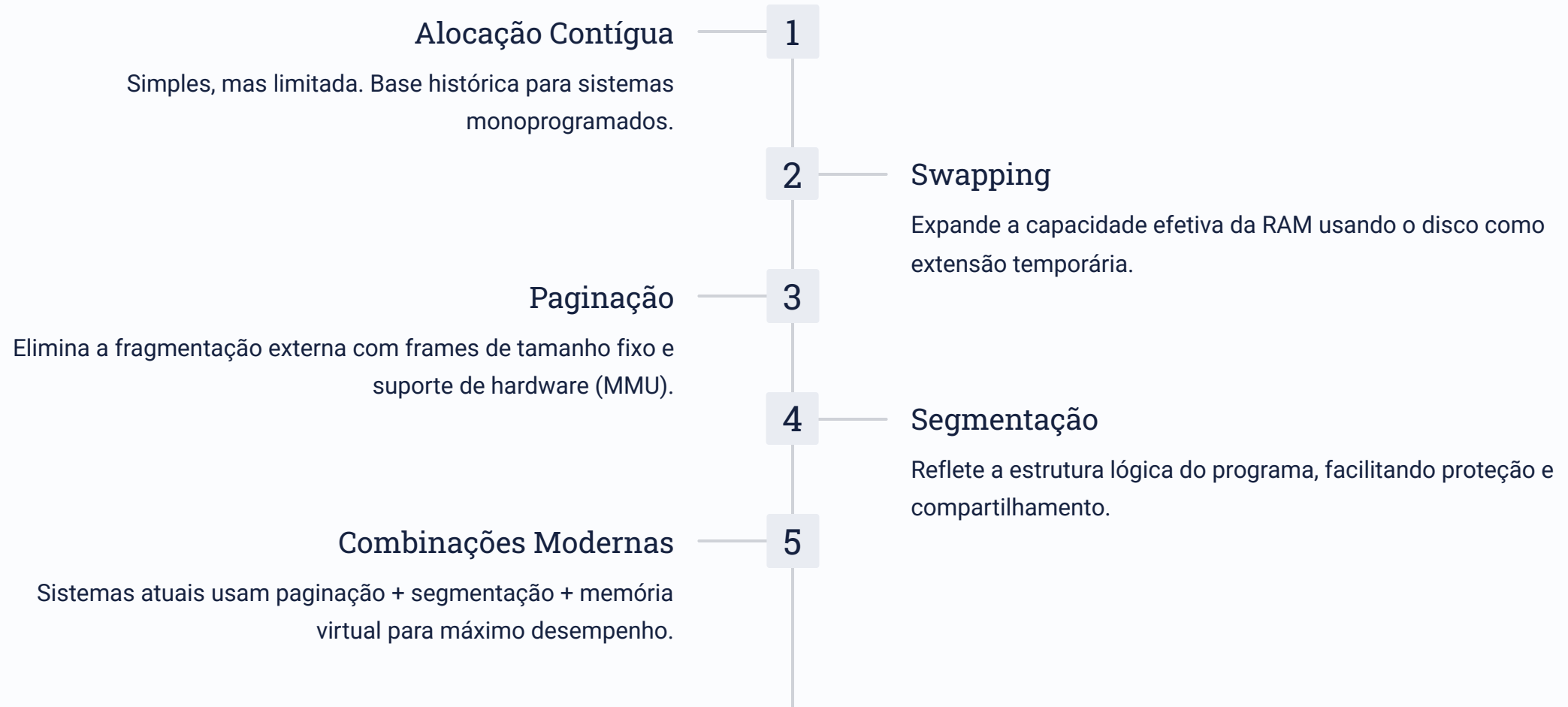


Proteção do Kernel

O SO opera em modo privilegiado (modo kernel), enquanto processos rodam em modo usuário. Isso protege as estruturas críticas do sistema operacional.

Gerenciamento de Memória: A Base para um Sistema Eficiente

A escolha da técnica de gerenciamento impacta diretamente o desempenho, a estabilidade e a segurança do sistema operacional.



  **Conclusão:** Não existe uma técnica universalmente superior. O SO moderno escolhe e combina estratégias com base nos requisitos de desempenho, segurança e arquitetura de hardware disponível.